CoCo Hands-On Lab: From Snowsight to CLI

Duration: 90 minutes
Level: Intermediate
Prerequisites:

- Snowflake account with CoCo enabled
- CoCo CLI installed
  - https://docs.snowflake.com/en/user-guide/cortex-code/cortex-code-cli
  - (`cortex --version` to verify)
- phData Toolkit Installed
  - Follow the steps in /lab/setup/toolkit_install.md
  - OR https://toolkit.phdata.io/docs/toolkit-cli#installation:~:text=contact%20us.-,Installation,-The%20Toolkit%20CLI
  - (`toolkit --version` to verify)

Overview

This hands-on lab teaches you to use **CoCo** in two environments to accelerate your development workflow:

Part	Environment	What You'll Do	Time
1	Snowsight (Web IDE)	Debug and extend a Python notebook that builds an ML model on sales data	30 min
2	CoCo CLI (Terminal)	Translate MS SQL Code into Snowflake SQL and dbt Models using phData Forge	45 min

---

Repository Structure

```
.
├── README.md                          # Lab guide and instructions
└── .cortex/
    ├── skills/
          ├── toolkit-configure
            └── SKILL.md   #toolkit configuration skill for CoCo
└── lab/
    ├── part1_notebook/
    │   └── sales_churn_model.ipynb    # Starter notebook with intentional
bugs
    └── part2_cli/
        └── medallion_financial_services.sql   # SQL script for migration
lab data
    ├── setup/
```

```
        └── setup.sql   # SQL Script for part 1 (if needed)
        └── toolkit_install.md   # Toolkit Installation readme
```

Setup

1. Clone the repo

```
git clone https://github.com/justindelisi-phdata/coco_hol.git
cd COCO_HOL
```

2. Run the SQL setup

(If needed, this should already be completed on your accounts) Execute `setup.sql` in your Snowflake account to create the required database, schema, and sample data.

3. Verify CoCo CLI

```
cortex --version
```

4. Verify phData Toolkit CLI

```
toolkit --version
```

Part 1: CoCo in Snowsight (45 min)

Context

Your team has a Python notebook that loads sales data, engineers features, and trains a model to predict whether a customer will churn. The notebook has several bugs and is missing key sections. You'll use CoCo in Snowsight to fix and extend it.

Setup (5 min)

1. Log into Snowsight
2. Navigate to **Projects > Notebooks**
3. Import the provided notebook: `lab/part1_notebook/sales_churn_model.ipynb`
4. Select the **Python 3 (Anaconda)** kernel and attach a warehouse

Exercise 0: Explain the Notebook (1 min)

Example prompt:

> *Can you summarize what this notebook does*

Exercise 1: Fix Errors and Get Best Practices (10 min)

After connecting to a compute pool, ask CoCo to review and fix the notebook.

Example prompt:

> *Can you search through this notebook and fix any errors you encounter. Additionally, please recommend any best practices*

CoCo will identify the intentional bugs in the notebook and suggest fixes, along with best practice recommendations for your ML workflow.

Exercise 2: Review and Run (5 min)

1. **Review the best practice recommendations** that CoCo provided — accept the ones that make sense for your use case
2. **Run All** cells in the notebook to verify everything executes cleanly

Exercise 3: Add Experimentation (15 min)

After a successful run, extend the notebook with advanced ML capabilities.

Example prompt:

> *Can you add Principal Component Analysis to the notebook. Additionally, I want to add Snowflake experiments across different ML algorithms to assess which model scores best*

This will add PCA for dimensionality reduction and set up experiment tracking to compare multiple models.

Exercise 4: Iterate and Fix (10 min)

1. **Ask CoCo to fix any remaining issues** — sometimes you'll need to append experiment names with timestamps depending on how many times you've run it
2. **Run All** again to verify the full pipeline executes end-to-end

What You've Accomplished

By the end of Part 1, you've taken a baseline model notebook, fixed errors, added robust experimentation across multiple ML algorithms, and set it up to deploy the best model to the Snowflake Model Registry each training run.

Part 2: CoCo CLI — Translating SQL with Forge Skills (45 min)

Context

Your team has a separate use case where they need to migrate an onprem SQL Server into Snowflake. We have the DDL to create the database and transformation layer, but it's in the MSSQL dialect.

`phData Forge` is an AI-native project delivery system that uses autonomous AI agents to automate code generation, testing, and legacy data migrations. By handling the heavy lifting of pipeline development, it shifts human data engineers into role-focused validators to compress project timelines by up to 70%

Setup (5 min)

1. Open your terminal
2. Navigate to your project directory:

```
cd COCO_HOL
```

3. Create a new toolkit instance within the directory:

```
toolkit init
```

4. Start CoCo:

```
cortex
```

5. Create a new connection:

- **First Time Opening Cortex Will Trigger the Set Up Wizard**
  - Follow the prompts to configure the connection.
- **Additional connections can be added by manually adding them to the configuration file:**

```
open -e ~/.snowflake/connections.toml
```

6. Verify your connection:

```
/status
```

Exercise 5: Translate MSSQL to Snowflake (30 min)

Step 1: List Skills

Example prompt:

> *Can you give me a couple sentence breakdown of the skills that are locally available to you?*

These are all the skills included with phData Forge.

Step 2: Explain the SQL Translate Skill

Example prompt:

> *What does the sql translate skill do?*

Explains about SQL translate

Step 3: Configure the phData Toolkit

Example prompt:

> *Use the toolkit-configure skill to configure the Toolkit with our current Snowflake connection*

Takes the current Snowflake connection for Cortex and uses it to configure the phData Toolkit

Step 4: Translate the Script

Example prompt:

> *Please use the sql translate skill to translate medallion_financial_services.sql from sql server to Snowflake, output it to a seperate file*

Step 5: Optimize the Script for Snowflake

Example prompt:

> *Identify and explain any performance issues the translated output file will have on snowflake, ranking them critical, high, medium and low priority*

Example prompt:

> *Please fix the critical and high priority performance issues*

Step 6: Validate the Script

Example prompt:

> *Run medallion_financial_services.sql in Snowflake using ds-exec skill to validate it runs , and fix any issues if any occur*

CoCo will run the scripts using the Toolkit execute skill and validate that there were no errors.

Exercise 6: Convert Stored Procedures to dbt Models (10 min)

Step 1: Convert Transformation Layer to dbt Models (switch back to UI)

Example prompt:

> *Stored procedures are too cumbersome, lets convert those transformation procedures in the financialservicesdb to dbt models*

Wrap-Up

What You Built

Component	Location	Purpose
Fixed & extended notebook	Snowsight	Debugged, added PCA, experimentation, and model registry deployment
Translated MSQL to Snowflake	Optimized SQL for Snowflake	Converted Stored Procedures to dbt Models

Key Takeaways

1. **CoCo in Snowsight** accelerates notebook development by helping you debug, generate, and explain code inline
2. **phData Forge** uses AI-native autonomous agents to automate complex data migrations, such as translating MS SQL Server scripts and stored procedures into optimized Snowflake SQL and dbt models, drastically compressing project timelines.

Next Steps

- Explore phData Forge: `https://www.phdata.io/phdata-forge/`
- Build custom skills to encode team expertise (see `$skill-development`)
- Define project rules with `AGENTS.md` for session-persistent conventions
- Try background agents: `Run a background agent to refactor all test files`
- Connect external tools via MCP: `cortex mcp add github -- npx @modelcontextprotocol/server-github`

Quick Reference

Action	Syntax
Include file	`@path/to/file`
Reference table	`#DB.SCHEMA.TABLE`
Invoke skill	`$skill-name`
Run shell command	`!git status`
Execute SQL	`/sql SELECT 1`
Switch mode	`Shift+Tab`
Plan mode	`Ctrl+P`
Help	`?`